

Jobsheet 14
Praktikum Algoritma dan Struktur Data

Nama: Dafa Naufal Rabbani
Kelas/No.Absen: TI-1G/05
NIM: 254107020086

1. Praktikum 1:

- Kode Program Mahasiswa05:
package Pertemuan14;

```
public class Mahasiswa05 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    public Mahasiswa05() {  
    }  
  
    public Mahasiswa05(String nim, String nama, String kelas, double  
ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
  
    public void tampilInformasi() {  
        System.out.println("NIM: " + this.nim + " " +  
"Nama: " + this.nama + " " +  
"Kelas: " + this.kelas + " " +  
"IPK: " + this.ipk);  
    }  
}
```

- Kode Program Node05:

```
package Pertemuan14;

public class Node05 {
    Mahasiswa05 mahasiswa;
    Node05 left, right;

    public Node05() {
    }

    public Node05(Mahasiswa05 mahasiswa) {
        this.mahasiswa = mahasiswa;
        left = right = null;
    }
}
```

- Kode Program BinaryTree05:

```
package Pertemuan14;

public class BinaryTree05 {
    Node05 root;

    public BinaryTree05() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa05 mahasiswa) {
        Node05 newNode = new Node05(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node05 current = root;
            Node05 parent = null;
            while (true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                }
                if (current == null) {
                    parent.left = newNode;
                }
            }
        }
    }
}
```

```

        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
}
}
}
}
}
}
}
}
}

```

```

public boolean find(double ipk) {
    boolean result = false;
    Node05 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }
    return result;
}
}

```

```

public void traversePreOrder(Node05 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}
}

```

```

public void traverseInOrder(Node05 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}
}

```

```

    }

    public void traversePostOrder(Node05 node) {
        if (node != null) {
            traversePostOrder(node.left);
            traversePostOrder(node.right);
            node.mahasiswa.tampilInformasi();
        }
    }

    public Node05 getSuccessor(Node05 del) {
        Node05 successor = del.right;
        Node05 successorParent = del;
        while (successor.left != null) {
            successorParent = successor;
            successor = successor.left;
        }
        if (successor != del.right) {
            successorParent.left = successor.right;
            successor.right = del.right;
        }
        return successor;
    }

    public void delete(double ipk) {
        if (isEmpty()) {
            System.out.println("Binary tree kosong");
            return;
        }
        Node05 parent = root;
        Node05 current = root;
        boolean isLeftChild = false;
        while (current != null) {
            if (current.mahasiswa.ipk == ipk) {
                break;
            } else if (ipk < current.mahasiswa.ipk) {
                parent = current;
                current = current.left;
                isLeftChild = true;
            } else if (ipk > current.mahasiswa.ipk) {
                parent = current;
                current = current.right;
                isLeftChild = false;
            }
        }
    }

```

```

    }
}
if (current == null) {
    System.out.println("Data tidak ditemukan");
    return;
} else {
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    } else if (current.left == null) {
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    } else if (current.right == null) {
        if (current == root) {
            root = current.left;
        } else {
            if (isLeftChild) {
                parent.left = current.left;
            } else {
                parent.right = current.left;
            }
        }
    } else {
        Node05 successor = getSuccessor(current);
        System.out.println("Jika 2 anak, current = ");
        successor.mahasiswa.tampilInformasi();
        if (current == root) {
            root = successor;
        } else {

```

```

        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}
}

```

- Kode Program BinaryTreeMain05:

```

package Pertemuan14;

public class BinaryTreeMain05 {
    public static void main(String[] args) {
        BinaryTree05 bst = new BinaryTree05();
        bst.add(new Mahasiswa05("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa05("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa05("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa05("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in order
traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54: ");
        String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak
ditemukan";
        System.out.println(hasilCari);

        System.out.print("Cari mahasiswa dengan ipk: 3.22: ");
        hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa05("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa05("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa05("244160170", "Fizi", "B", 3.46));
        System.out.println("\nDaftar semua mahasiswa setelah penambahan
3 mahasiswa:");
        System.out.println("InOrder Traversal:");
    }
}

```

```

        bst.traverseInOrder(bst.root);
        System.out.println("\nPreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("\nPostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("\nPenghapusan data mahasiswa");
        bst.delete(3.57);
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan
1 mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);
    }
}

```

- Hasil Running:

```

PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData>
cp' 'C:\Users\Dapaa\AppData\Roaming\Code\User\workspaceStorage\faaac69
'Pertemuan14.BinaryTreeMain05'

```

```

Daftar semua mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

```

```

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54: Ditemukan
Cari mahasiswa dengan ipk: 3.22: Tidak ditemukan

```

```

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

```

```

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

```

```

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

```

```

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

```

```

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData>

```

- Jawaban Pertanyaan:
 1. Karena Binary Search Tree (BST) memiliki aturan penempatan data yang terstruktur, di mana semua node di subtree kiri memiliki nilai yang lebih kecil dari root, dan semua node di subtree kanan memiliki nilai yang lebih besar.
 2. Atribut left digunakan untuk menyimpan referensi atau pointer ke node anak sebelah kiri (child node yang nilainya lebih kecil). Atribut right digunakan untuk menyimpan referensi atau pointer ke node anak sebelah kanan (child node yang nilainya lebih besar atau sama dengan).
 3. Jawaban:
 - a. Atribut root berfungsi sebagai pintu masuk utama atau fondasi dari struktur data pohon. Semua operasi traversal, pencarian, penambahan, dan penghapusan harus dimulai dari node root ini.
 - b. Ketika objek tree pertama kali dibuat melalui konstruktor, nilai dari root adalah null.
 4. Node baru tersebut akan langsung dialokasikan ke variabel root. Pohon yang tadinya kosong sekarang akan menjadikan node baru tersebut sebagai node puncak utamanya.
 5. Baris program tersebut digunakan untuk proses penelusuran (traversal) ke bawah pohon secara iteratif guna mencari lokasi kosong yang tepat untuk menyisipkan node baru. Variabel parent menyimpan posisi node saat ini sebelum melangkah ke node anak (left atau right) agar saat mencapai posisi kosong (null), sistem tetap memiliki referensi ke node atasnya untuk menyambungkan node baru tersebut menggunakan perintah `parent.left = newNode` atau `parent.right = newNode`.
 6. Langkah saat menghapus node dengan dua anak:
 - 1) Cari node pengganti (successor) dengan memanggil method `getSuccessor()`. Successor ini adalah node dengan nilai terkecil di dalam subtree sebelah kanan dari node yang akan dihapus.
 - 2) Hubungkan node successor tersebut untuk menempati posisi node yang dihapus dengan mengatur pointer milik node parent dari node yang dihapus agar menunjuk ke successor.
 - 3) Sambungkan subtree bagian kiri dari node lama yang dihapus ke pointer sebelah kiri milik successor (`successor.left = current.left`).

2. Praktikum 2:

- Kode Program BinaryTreeArray05:
package Pertemuan14;

```
public class BinaryTreeArray05 {
    Mahasiswa05[] dataMahasiswa;
    int idxLast;

    public BinaryTreeArray05() {
        this.dataMahasiswa = new Mahasiswa05[10];
    }

    void populateData(Mahasiswa05 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2 * idxStart + 1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }
}
```

- Kode Program BinaryTreeArrayMain05:
package Pertemuan14;

```
public class BinaryTreeArrayMain05 {
    public static void main(String[] args) {
        BinaryTreeArray05 bta = new BinaryTreeArray05();
        Mahasiswa05 mhs1 = new Mahasiswa05("244160121", "Ali", "A",
3.57);
        Mahasiswa05 mhs2 = new Mahasiswa05("244160185", "Candra",
"C", 3.41);
        Mahasiswa05 mhs3 = new Mahasiswa05("244160221", "Badar",
"B", 3.75);
        Mahasiswa05 mhs4 = new Mahasiswa05("244160220", "Dewi",
"B", 3.35);
    }
}
```

```

        Mahasiswa05 mhs5 = new Mahasiswa05("244160131", "Devi",
"A", 3.48);
        Mahasiswa05 mhs6 = new Mahasiswa05("244160205", "Ehsan",
"D", 3.61);
        Mahasiswa05 mhs7 = new Mahasiswa05("244160170", "Fizi", "B",
3.86);

        Mahasiswa05[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4,
mhs5, mhs6, mhs7, null, null, null};
        int idxLast = 6;
        bta.populateData(dataMahasiswas, idxLast);
        System.out.println("\nInorder Traversal Mahasiswa: ");
        bta.traverseInOrder(0);
    }
}

```

- Hasil Running:

```

PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData> &
cp' 'C:\Users\Dapaa\AppData\Roaming\Code\User\workspaceStorage\faaac69d0
'Pertemuan14.BinaryTreeArrayMain05'

Inorder Traversal Mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData>

```

Jawaban Pertanyaan:

1. Atribut dataMahasiswa (array) digunakan untuk wadah penyimpanan elemen atau objek node dari pohon biner secara sekuensial. Atribut idxLast berfungsi mencatat batas posisi indeks terakhir yang berisi data di dalam array tersebut agar membatasi perulangan rekursif.
2. Method populateData() digunakan untuk memasukkan sekumpulan data array objek mahasiswa beserta nilai batas indeks terakhir ke dalam atribut class BinaryTreeArray secara instan.
3. Method traverseInOrder() digunakan untuk menelusuri elemen-elemen pohon biner yang berada di dalam array dan mencetaknya ke layar dengan urutan Left-Root-Right secara rekursif.
4. Jika indeks root atau parent berada di $i = 2$:
 - Left child: $2 \times i + 1 = 2 \times 2 + 1 = \mathbf{5}$
 - Right child: $2 \times i + 2 = 2 \times 2 + 2 = \mathbf{6}$
5. Pernyataan `int idxLast = 6` memberikan informasi kepada sistem pohon biner array bahwa data mahasiswa yang valid tersimpan dari indeks ke-0 hingga indeks ke-6, sehingga proses penelusuran rekursif tahu kapan harus berhenti mencari elemen anak baru.
6. Rumus $2 \times \text{idxStart} + 1$ dan $2 \times \text{idxStart} + 2$ merupakan representasi matematis standar untuk memetakan hubungan hierarki parent-child dari Complete Binary Tree ke dalam representasi linear berbasis indeks Array 0-indexed. Melalui skema perkalian ini, struktur pohon tetap terjaga tanpa membutuhkan pointer fisik objek tautan.